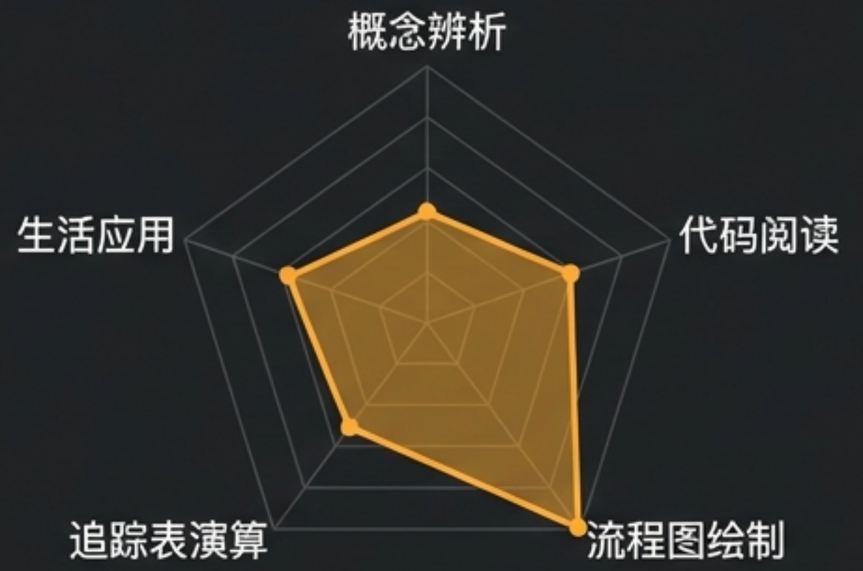
第一册 第2章 演算法与程式设计

统考冲刺 [Tier 1 绝对核心考区]

基于《第一册 02演算法与程式设计.pdf》极致浓缩

Exam Radar



HUD Console

```
1  // 本章核心执行逻辑
2  人类逻辑构思 -> 图形与虚拟码构建 -> Python 语法翻译 -> 机器执行与除错
```

Gemini Notebook

核心执行指令（TODO_LIST）

【解析 Parse】

解构解决问题的4大步骤，区分人类逻辑与电脑执行的本质差异。

【构建 Build】

熟练绘制标准流程图，并能编写逻辑严密的虚拟码（Pseudocode）。

【编码 Code】

掌握 Python 核心语法体系，精准应用输出指令与算术运算符优先级。

【追踪 Trace】

驾驭三大控制结构，精准执行代码单步演算（Dry Run）与除错（Debug）。

Gemini Notebook

>_ System Architecture

- 2.1 解決问题的过程
  - 一般过程 (分析 -> 寻找 -> 解决与验证)
  - 电脑解决过程 (分析 -> 设计方案 -> 编写程式 -> 调试程式)
  - 【! Debug 环节】
- 2.2 演算法与表达方式
  - 5大核心特征 (输入、输出、明确性、有限性、可行性)
  - 表达双轨制 (流程图 Flowchart vs 虚拟码 Pseudocode)
- 2.3 程式设计基本知识
  - Python 语法基建 (注释、print、算术运算)
  - 三大控制结构引擎 (顺序、选择、重复)

Gemini Notebook

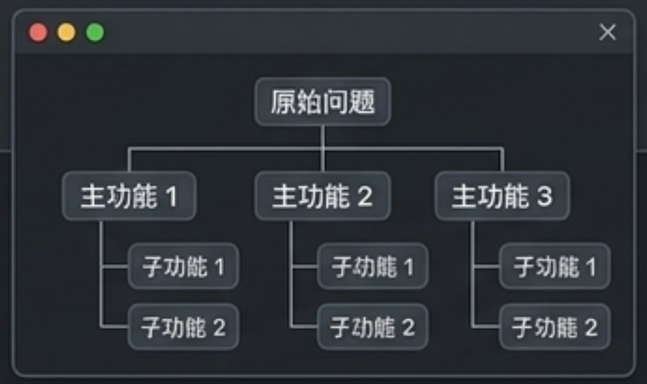
核心执行指令（TODO_LIST）- Human vs. Computer Problem Solving Pipelines

人类解题引擎 (Human Problem Solving Engine)

分析问题
(Analyze Problem)
→
寻找解决方案
(Find Solution)
→
解决问题与验证结果
(Solve & Verify Result)

电脑解题引擎 (Computer Problem Solving Engine)

分析问题
(Analyze Problem)
→
设计方案
(Design Solution - Divide & Conquer)
→
编写程式
(Write Code)
→
调试程式
(Debugging)



⚠ Bugs 出没区：语法错误、逻辑错误、运行时错误

Gemini Notebook

演算法 5 大刚性 DNA



表达方式大比拼

流程图 (Flow Chart)

图形化表达、结构清楚、
直观易懂、适合初学者



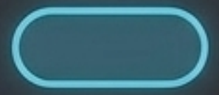
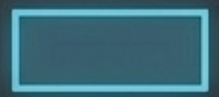
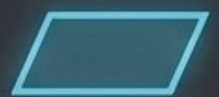
虚拟码 (Pseudocode)

类代码逻辑、
无严格语法限制、
依靠缩进对齐判定层级



Gemini Notebook

逻辑元素图谱

	开始/结束（流程的起止）
	处理（执行运算或程式）
	输入/输出（读取或打印数据）
	判断（根据条件决定流向，必须标注 True/False）
	连接点（用于跨页或断开的流程线）

⚠ 统考制图底线原则

- 必须使用标准符号
- 绘制方向：由上至下、由左至右
- 判断框必须是单入双出

Gemini Notebook

指令台 print()

```
print("马来西亚", "boleh!")
```

```
马来西亚 boleh!
```

⚠️ 必须全小写！标点符号(引号、括号、逗号)必须是英文半角！

隐身涂层（注释规范）

单行注释：`# 直译器会忽略 # 后的内容`

多行注释：前后三个单/双引号包裹 `''' 注释内容 '''`

缩进法则（Indentation）

Python 的灵魂！使用相同的缩进表示同一语句块（Statement Block）。

```
if True:
    print("This is indented.")
```

Gemini Notebook

Mathematical Operators Pyramid (Order of Precedence)



⚠️ 高频致命考点拆解（The Division Trio）

/ （浮点除法）

8 / 2 结果是 4.0（即使是整数相除，结果也是浮点数）

// （整除/向下取整）

7 // 2 结果是 3；
-7 // 2 结果是 -4 （极易错点！向下取小）

% （取模/求余数）

7 % 2 结果是 1

⚠️ **ZeroDivisionError 警告：分母绝对不能为 0！**

Gemini Notebook

控制结构 I：分支轨道（Selection）



单向 if

只处理条件成立的情况，不成立直接略过。



注意 else：必须加冒号。

双向 if-else

非黑即白，两边必走其一。



多分支 if-elif-else

层层筛查，一旦某个 elif 成立，执行完毕立刻跳出整个结构。



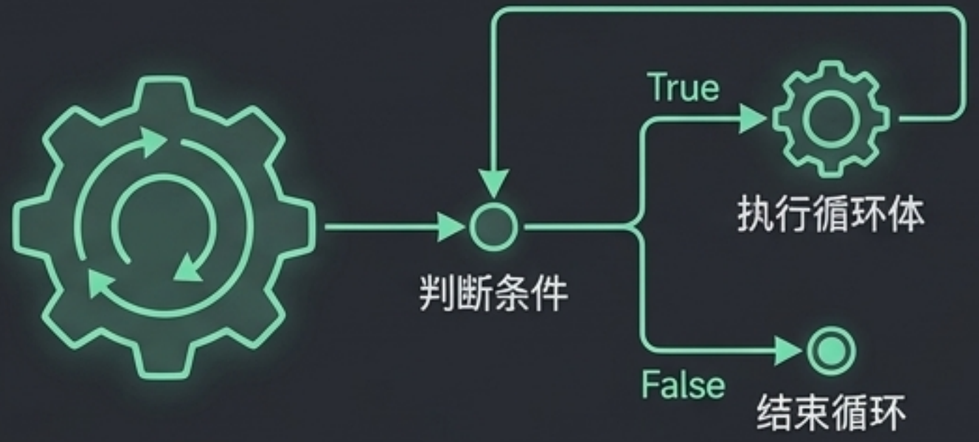
嵌套（Nested if）

套娃逻辑，在条件成立的内部再设关卡，严格依靠缩进分层。

Gemini Notebook

控制结构 II：重复执行引擎（Iteration）

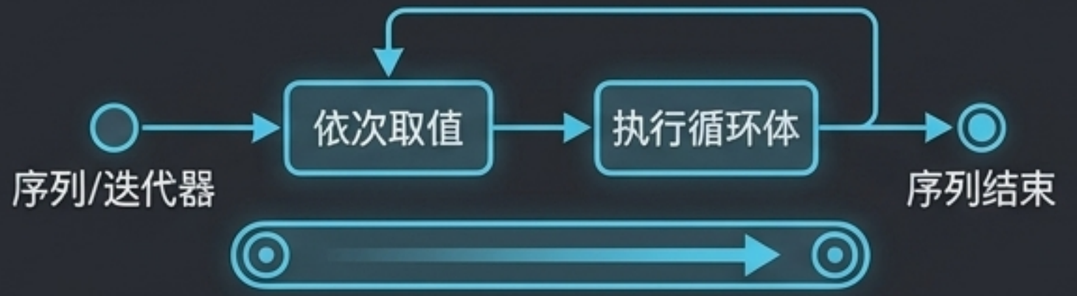
条件驱动引擎：while 循环



逻辑：先判断条件，为 True 则执行，执行完再次判断，直到 False 结束。

⚠ 必须要有改变条件的语句（如 i += 1），否则陷入死循环（Infinite Loop）。

序列驱动引擎：for 循环与 range()



逻辑：依次从序列中取值。

range() 函数示例	产生序列	说明
range(6)	0，1，2，3，4，5	默认从0开始，终值 - 1
range(-3，3)	-3，-2，-1，0，1，2	起始值到终值 - 1
range(1，6，2)	1，3，5	步长/间隔为2

Gemini Notebook

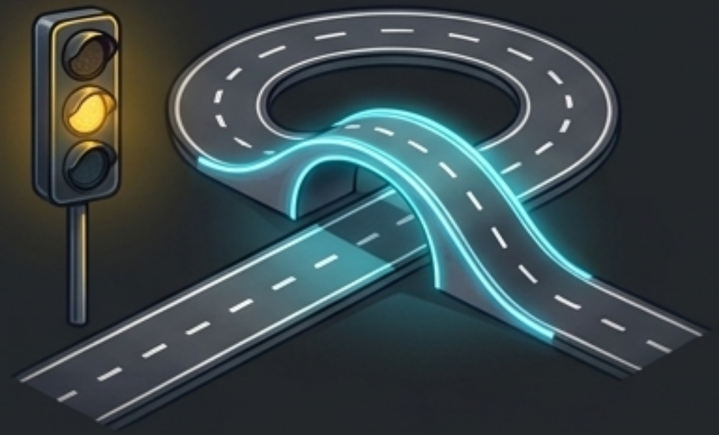
循环控制截断（Flow Disruption）



break 语句（彻底摧毁）

立刻强制跳出现有的整个循环，继续执行循环体外部的后续语句。

```
if city == 'x':
    break   # 直接结束全剧
```



continue 语句（跳跃重置）

跳过本次循环体中剩余的语句，直接返回循环顶部，判断条件进入下一轮！

```
if i % 5 == 0:
    continue  # 忽略本次剩余，重开头再来
```

Gemini Notebook

高频混淆架构对决

For 循环 VS While 循环

For 循环

明确知道循环次数、遍历序列时首选。

```
for i in range(n):
```

While 循环

循环次数未知，必须依赖特定条件触发结束时首选。

```
while condition:
```

除法三兄弟的硬核算术对决

`7 / 2` = 3.5 (Float)　　`7 // 2` = 3 (Int 向下取整)　　`7 % 2` = 1 (Int 余数)

流程图 VS 虚拟码

流程图

图形化沟通，全局结构一目了然。



虚拟码

类代码书写，方便直接向 Python 程式过渡。

```
如果 条件 成立：执行操作
否则：其他操作
```

Gemini Notebook

追踪表单步演算实战（Dry Run）

代码视窗

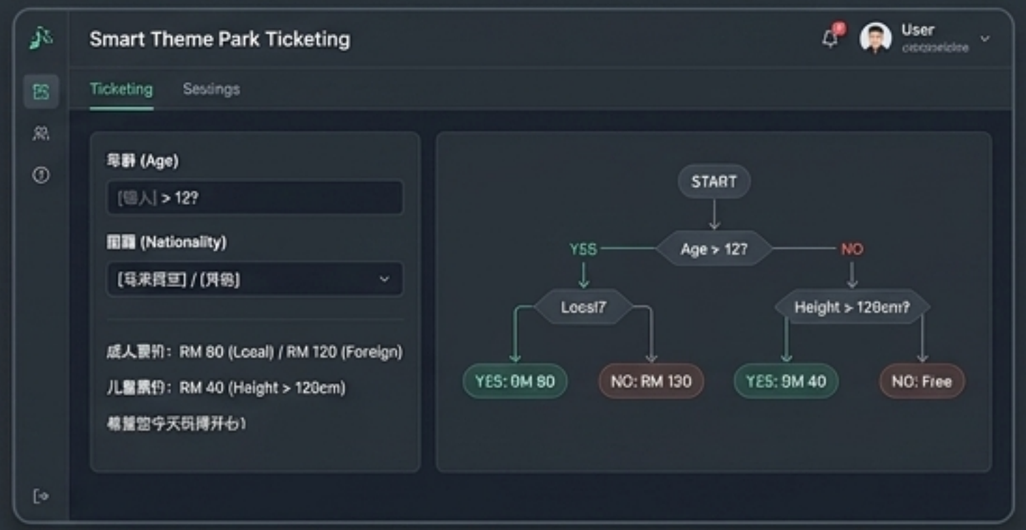
```
balance = 100
year = 1
while year <= 3:
    balance = balance + (balance * 0.03)
    year = year + 1
```

动态追踪表 (Trace Table)

Iteration	Condition	year	balance
Init		year = 1	100
1	year <= 3 (True)	103	year = 2
2	year <= 3 (True)	106.09	year = 3
3	year <= 3 (True)	109.27	year = 4
Exit	year <= 3 (False)	-> 循环结束！最终 year 值是 4	

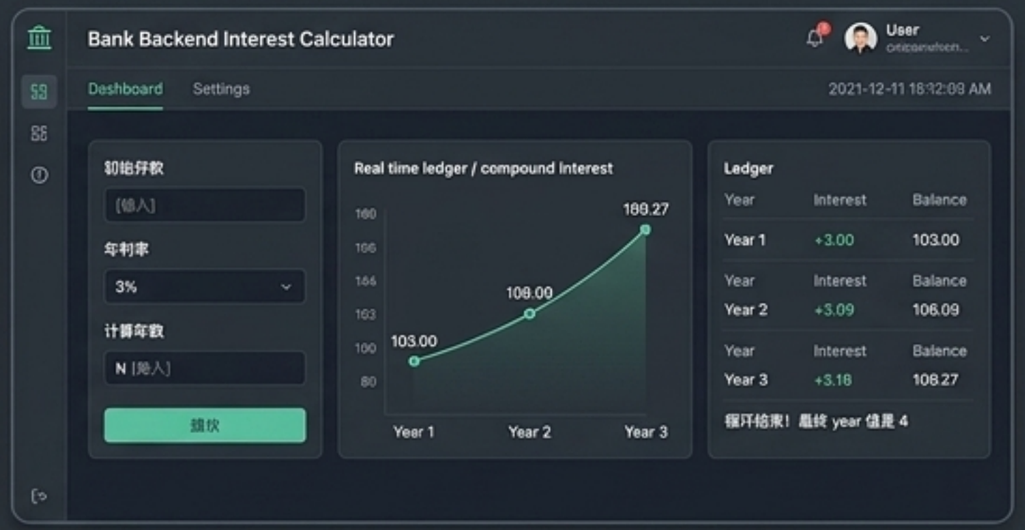
Gemini Notebook

情境 A：主题乐园智能闸机




```
1  业务需求：根据年龄(>12)与国籍双重判定票价。
2
3  技术选型：嵌套 if-else（Nested Selection）。
4
5  逻辑树：第一层拦截年龄，第二层核实国籍，多维
   度复合决策。
```

情境 B：银行后台利息结算系统




```
1  业务需求：连续计算未来 N 年的复利滚存。
2
3  技术选型：while 循环引擎（Iteration）。
4
5  逻辑树：初始存款打底，年复一年循环计算，直到
   满足年份条件精准停机。
```

Gemini Notebook

☠️ 统考致命失分 Bugs（Fatal Errors）

☠️ 1. 忘记在 if，else，while，for 语句末尾加冒号：！（高频！）
⚠️ 2. 代码中混入中文标点（尤其是引号“ ”和括号（ ））。
⚠️ 3. 缩进（Indentation）参差不齐，导致层级彻底崩坏。
⚠️ 4. 误以为 range(1，10) 会包含 10，其实终值要减一，只算到 9!
⚠️ 5. print() 首字母大写成 Print()（大小写严格敏感）。

🏆 满分速查口诀（Golden Rules）

算法五特征：输、出、明、有、可

画流程图：上下左右，单入双出，字简图明

算术先决：幂顶层，乘除紧随，加减垫底

Gemini Notebook